

EventForce Management System

Phase 1: Requirement Analysis and Planning

Implementation review: 3 October 2026 | Dedicated project playground

Problem and user story

Event coordinators need one system for client requests, venue availability, vendor assignments and feedback. We designed EventForce to reduce scattered booking information and make cancellation decisions traceable.

Ideation and customer journey

The design persona prioritizes accurate bookings and clear ownership. The journey is client registration, event planning, venue selection, vendor assignment, confirmation, event execution and feedback. Cancellation branches into approval or rejection. This is a design model, not an interview study.

Functional requirements

Maintain Event, Client, Vendor, Venue, Feedback and Event Vendor records; prevent venue/date conflicts; support cancellation approval; send cancellation and reminder notifications; complete past events on a schedule; restrict access by responsibility; report on upcoming events.

Nonfunctional requirements

Preserve related-record integrity, validate required booking information, support bulk Apex operations and provide understandable errors. No production performance benchmark or uptime guarantee has been measured.

Architecture and data model

Lightning app and custom tabs form the interface. Event looks up Client and Venue. Feedback looks up Event and Client. Event Vendor links Event and Vendor through two master-detail relationships. Flows, approvals and Apex provide automation; reports and dashboards provide operational views.

Plan and acceptance criteria

Build the data model first, then booking logic, cancellation, reminders, UI, sample data, security and regression tests. Evidence must distinguish saved configuration from runtime outcomes. Inbox delivery and separate role sessions remain final acceptance checks.